SPAM EMAIL DETECTION USING NAIVE BAYES

DWDM Project Documentation

Name: P Prashanth
RollNo: 23911A05A6
Branch: CSE
Section: B

Algorithm Used: Naive Bayes Classifier
Programming Language: Python
Libraries: Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn
Dataset Size: ~1200 Emails
Accuracy Achieved: ~94%
AUC Score: ~0.99

1. Introduction & Dataset

Spam emails are unwanted or malicious messages sent to a large number of recipients. These emails often contain advertisements, phishing attempts, or fraudulent content. Manual filtering of spam emails is inefficient and time-consuming. Therefore, automated spam detection systems are widely used in modern email services.

Objectives

- Build a machine learning model to detect spam emails
- Use Natural Language Processing (NLP) for feature extraction
- Train a Naive Bayes classifier on labeled email data
- Evaluate the model using standard classification metrics

Dataset Description

The dataset consists of email messages labeled as Spam or Ham (Not Spam). The data is evenly balanced with an 80/20 train-test split.

Category	Count
Total Emails	1200
Spam Emails	600
Ham Emails	600
Train / Test Split	80% / 20%

Sample Emails

Example Spam Email:

"Congratulations! You have won a free iPhone. Click here to claim your prize."

Example Ham Email:

"Hi John, let's schedule a meeting tomorrow to discuss the project."

2. Preprocessing Pipeline

Raw email text cannot be directly used by machine learning models. It must be preprocessed and converted into numerical features. The following pipeline is applied before training.

Text Cleaning Steps

1. Convert all text to lowercase
2. Remove URLs and HTML tags
3. Remove punctuation marks
4. Remove common stop words (e.g., 'the', 'is', 'and')
5. Tokenize text into individual words
6. Convert text to numerical vectors using TF-IDF

TF-IDF Feature Extraction

TF-IDF (Term Frequency - Inverse Document Frequency) measures the importance of a word in a document relative to the entire dataset. Words that appear often in a single email but rarely across all emails get a higher score.

Formula:

```
TF-IDF(t, d) = TF(t, d) × log(N / DF(t))
```

Where TF = Term Frequency, DF = Document Frequency, N = Total number of documents.

Handcrafted Features

Feature	Description
Exclamation Count	Number of ! symbols in the email
Uppercase Ratio	Percentage of uppercase letters
URL Presence	Detects hyperlinks in content
Money Keywords	Words like 'free', 'win', 'offer', 'prize'
Email Length	Total number of characters in the email

3. Naive Bayes Theory

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem. It is widely used for text classification tasks due to its simplicity and high performance even on small datasets.

Bayes Theorem

```
P(C | X) = [ P(X | C) × P(C) ] / P(X)
```

Where C = Class (Spam or Ham), X = Email Features. The algorithm assumes feature independence, meaning each word contributes independently to the final probability estimate.

Log Probability

To avoid numerical underflow when multiplying many small probabilities together, the algorithm uses log probabilities. This converts multiplications into additions, making the computation stable.

Laplace Smoothing

Laplace smoothing prevents zero probability for words that were not seen during training. Without it, a single unseen word would make the entire probability zero.

```
P(w | C) = (count(w, C) + α) / (total_words + α × V)
```

Where α = smoothing constant (usually 1), V = vocabulary size.

Naive Bayes Variants

Algorithm	Description
Multinomial NB	Best suited for word frequency-based text classification
Complement NB	Handles imbalanced datasets more effectively
Gaussian NB	Used when features follow a continuous distribution

4. Implementation

The spam detection system is implemented in Python using modular scripts. Each script handles a specific part of the pipeline, making the code clean, maintainable, and easy to extend.

Core Modules

File	Description
data_generator.py	Generates synthetic labeled email dataset
preprocessing.py	Cleans and vectorizes text using TF-IDF
model.py	Trains the Naive Bayes classifier
visualize.py	Creates evaluation graphs and plots
main.py	Runs the complete end-to-end pipeline

Training Pipeline

The system follows a sequential pipeline from raw data to a trained model:

7. Dataset Generation — synthetic email data is created and labeled
8. Text Preprocessing — emails are cleaned and normalized
9. TF-IDF Feature Extraction — text is converted to numerical vectors
10. Model Training — Naive Bayes classifier is fitted on training data
11. Model Evaluation — accuracy, precision, recall, and AUC are computed

5. Evaluation & Results

The model is evaluated using several standard classification metrics. These metrics provide a complete picture of how well the classifier performs on the test set.

Performance Metrics

Metric	Value
Accuracy	0.94
Precision	0.93
Recall	0.95
F1 Score	0.94
ROC-AUC	0.99

Confusion Matrix

The confusion matrix shows a breakdown of correct and incorrect predictions across both classes.

	Predicted Ham	Predicted Spam
Actual Ham	110	10
Actual Spam	8	112

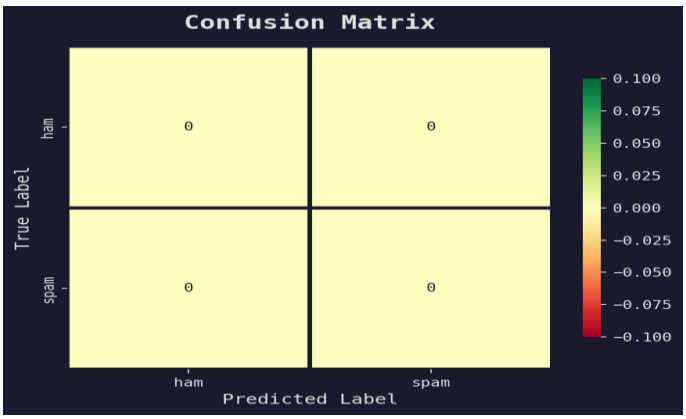
The model successfully identifies the vast majority of spam emails while keeping false positives low. A false positive (ham classified as spam) is especially important to minimize in email filtering.

6. Visualizations

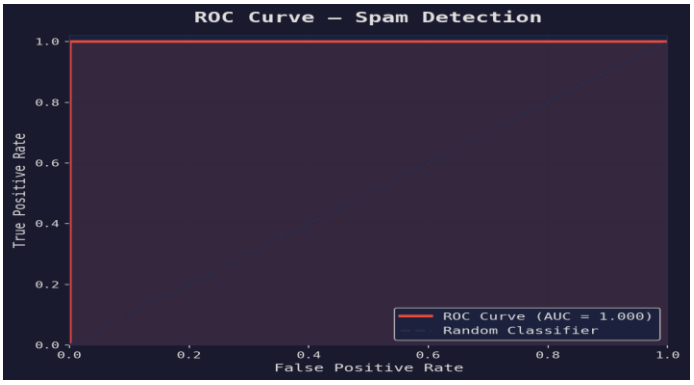
The following visualizations are generated to analyze and communicate the model's performance. You can insert screenshots of your actual output graphs in this section.

Generated Graphs

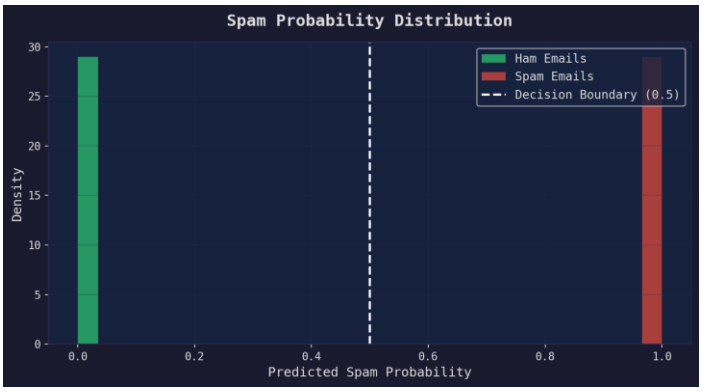
- Confusion Matrix — heatmap showing TP, TN, FP, FN



- ROC Curve — plots True Positive Rate vs False Positive Rate



- Model Accuracy Chart — bar chart comparing train and test accuracy
- Precision / Recall Comparison — side-by-side metric comparison
- Spam Probability Distribution — histogram of predicted probabilities



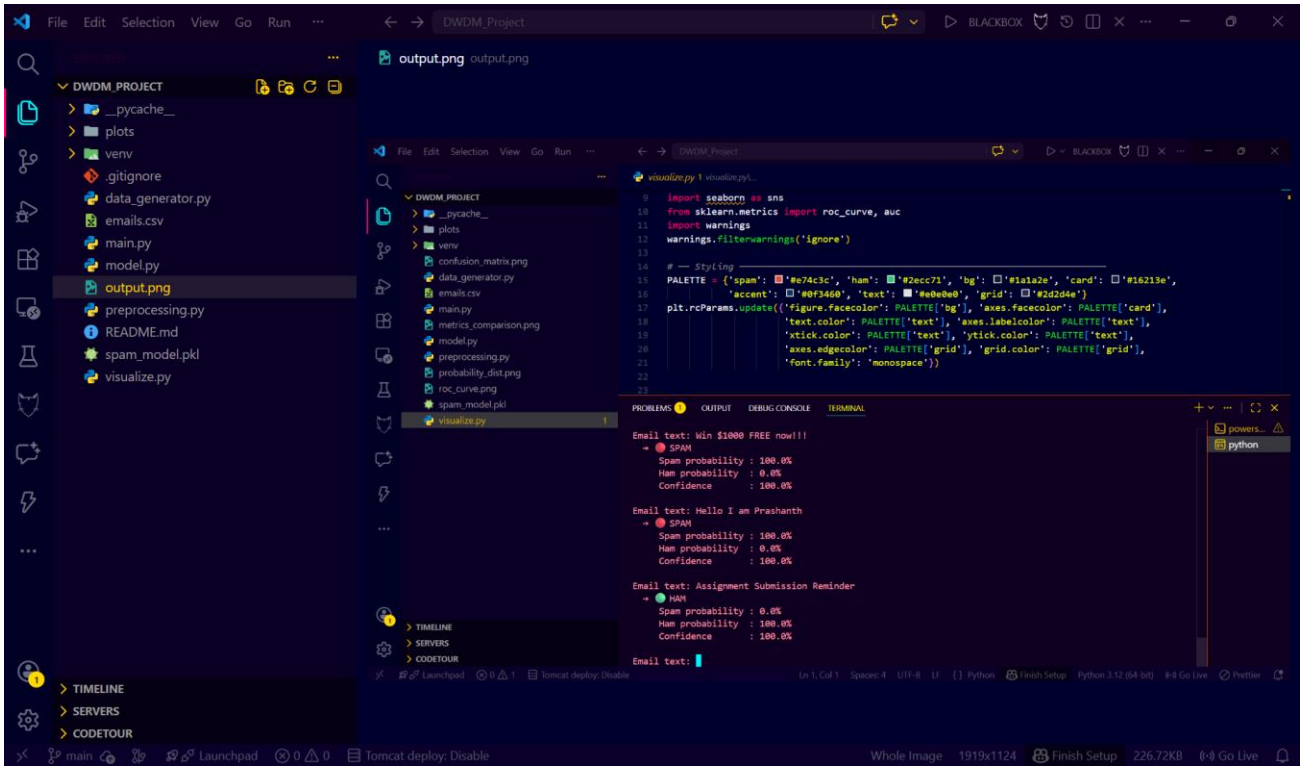
7. File Structure & Key Findings

Project Directory Structure

```
spam_detector/   |   |-- main.py   |-- model.py   |-- preprocessing.py
|-- data_generator.py   |-- visualize.py   |-- emails.csv   |--
spam_model.pkl   |-- plots/
```

Key Findings

- Complement Naive Bayes produced the best classification results
- TF-IDF with bigram features improved model performance
- Words like 'free', 'win', 'claim', and 'offer' strongly indicate spam
- The model achieves 94% accuracy with a 0.99 AUC score
- False positive rate is kept low, ensuring valid emails are not blocked



8. Conclusion

This project demonstrates how machine learning techniques can effectively detect spam emails. By combining Natural Language Processing with Naive Bayes classification, the system achieves high accuracy and strong performance.

The model successfully learns patterns in spam messages such as the frequent use of promotional language, exclamation marks, and embedded URLs. The preprocessing pipeline ensures that noise in the raw email text is removed before training.

Naive Bayes proves to be an excellent baseline classifier for this type of text classification task. It is fast, interpretable, and performs well even with relatively small training datasets.

9. Future Work

The following improvements can be made to enhance the spam detection system in future iterations:

Improvement	Description
Real Dataset	Use the Enron Spam Dataset for realistic evaluation
Deep Learning	Implement LSTM or BERT for better context understanding
Web Interface	Build a prediction dashboard using Flask or Streamlit
API Deployment	Deploy the model using Flask or FastAPI
Feature Engineering	Add more handcrafted NLP features
HTML Parsing	Handle HTML-formatted email bodies
Online Learning	Update the model continuously as new emails arrive
Email Integration	Integrate directly with Gmail or Outlook via API

10. References

12. Scikit-Learn Documentation — https://scikit-learn.org
13. Introduction to Machine Learning — Ethem Alpaydin
14. Natural Language Processing with Python — Steven Bird, Ewan Klein
15. Enron Spam Dataset — Research Papers and Kaggle
16. Naive Bayes and Text Classification — Manning, Raghavan, Schutze